

O que um controller de API REST deve fazer?

Em uma API REST, o **controller** é uma peça que fica na entrada do sistema. Ele recebe a requisição HTTP que vem do cliente, interpreta os dados enviados, chama a parte responsável pela lógica da aplicação e devolve uma resposta HTTP.

Em um exemplo simples de autenticação, o controller pode receber um login e uma senha, encaminhar esses dados para o caso de uso de autenticação e, depois, devolver ao cliente uma resposta dizendo se o login deu certo ou não.

Neste projeto, a ideia geral é que controllers sejam finos, focados em orquestração, e que as regras de negócio não fiquem dentro deles. Também se espera separação clara entre responsabilidades e comportamento previsível da API.

Papel do controller, em termos simples

Podemos pensar no controller como um “atendente” da API:

- ele **recebe** a requisição;
- ele **confere** se os dados básicos vieram corretamente;
- ele **passa** esses dados para a parte do sistema que realmente decide o que fazer;
- ele **devolve** a resposta no formato HTTP.

Portanto, o controller não deve ser o lugar principal da inteligência do sistema. Ele deve ser apenas a camada de entrada.

O básico que um controller deve observar

1. Receber corretamente a requisição

A primeira responsabilidade do controller é receber os dados enviados pelo cliente.

Em autenticação, isso normalmente significa receber algo como:

- email e senha;
- ou login e senha;
- ou token de atualização, em um endpoint de renovação de sessão.

O controller deve saber de onde esses dados vêm:

- do corpo da requisição;
- dos headers;
- dos cookies;
- ou dos parâmetros da URL, dependendo do caso.

O mais importante é entender que o controller é o ponto onde os dados chegam ao sistema.

2. Fazer validações simples de entrada

O controller deve verificar o básico antes de continuar.

Por exemplo:

- o campo `email` foi enviado?
- o campo `password` foi enviado?
- esses campos são texto?
- o corpo da requisição está em um formato aceitável?

Essas validações são simples e pertencem à borda da aplicação.

Exemplos do que o controller pode verificar:

- campo obrigatório ausente;
- tipo inválido;
- corpo vazio;
- formato claramente inadequado.

Exemplos do que ele não deve decidir sozinho:

- se a senha está correta;
- se o usuário está bloqueado;
- se o token foi revogado;
- se aquele usuário realmente pode acessar o sistema.

Essas decisões pertencem à lógica da aplicação, não ao controller.

3. Chamar a parte certa do sistema

Depois de receber e validar o básico, o controller deve chamar o serviço, caso de uso ou componente responsável pela autenticação.

Ou seja, o controller não “resolve tudo sozinho”. Ele delega o trabalho principal.

Em uma arquitetura bem organizada, isso é importante porque:

- deixa o código mais limpo;
- facilita testes;
- evita misturar HTTP com regra de negócio;
- facilita manutenção futura.

4. Retornar uma resposta HTTP clara

Depois que a lógica de autenticação for executada, o controller precisa devolver uma resposta compreensível para o cliente.

Exemplos comuns:

- autenticação bem-sucedida;
- credenciais inválidas;
- dados obrigatórios ausentes;
- erro interno inesperado.

O importante, para começo de aprendizagem, é que a resposta seja:

- coerente;
- simples;
- previsível;
- organizada.

Por exemplo, não é uma boa prática retornar mensagens completamente diferentes para situações parecidas sem necessidade.

5. Não colocar muita lógica dentro do controller

Esse é um dos pontos mais importantes.

Mesmo em exemplos simples, é recomendável evitar que o controller vire um lugar onde tudo acontece.

Não é interessante que ele:

- consulte banco diretamente;
- aplique regra de negócio complexa;
- tome várias decisões do domínio;
- misture autenticação, autorização, logs, persistência e montagem de resposta em um único bloco.

A ideia prática pode ser resumida assim:

o controller recebe, organiza e encaminha; quem decide as regras do negócio é outra parte do sistema.

6. Ter cuidado com segurança básica

Como autenticação lida com dados sensíveis, o controller precisa tomar alguns cuidados simples, mesmo em um exemplo inicial.

Cuidados essenciais

- não expor a senha em logs;
- não devolver mensagens com detalhes excessivos;
- não mostrar erros internos do sistema diretamente para o usuário;
- tratar entradas externas como não confiáveis.

Por exemplo, em vez de devolver algo como:

“usuário não encontrado no banco X”

é melhor algo mais simples, como:

“credenciais inválidas”

Isso evita exposição desnecessária do comportamento interno do sistema.

7. Manter o código compreensível

Como este é um exemplo inicial, o mais importante é que o controller seja:

- pequeno;
- legível;
- bem nomeado;
- fácil de acompanhar.

Neste momento, não é necessário exagerar em sofisticação arquitetural. O essencial é ensinar corretamente o papel do controller e evitar erros conceituais básicos.

O que um controller inicial de autenticação deve fazer

Em um cenário simples de login, o controller deve:

1. receber email e senha;
2. verificar se esses dados foram enviados;
3. chamar a lógica de autenticação;
4. receber o resultado;
5. devolver a resposta HTTP correspondente.

Esse fluxo já é suficiente para um primeiro exemplo didático.

O que ele não deve fazer

Para não confundir o papel do controller, é melhor evitar que ele:

- implemente a verificação real da senha;
 - consulte diretamente o banco de dados;
 - gere regras de permissão misturadas no mesmo método;
 - faça muita lógica condicional complexa;
 - concentre toda a inteligência da autenticação.
-

Resumo

O controller é a porta de entrada da API.

Ele deve:

- receber a requisição;
- validar o básico;
- chamar a lógica certa;
- devolver a resposta.

Ele não deve virar o “coração do sistema”.

Para um projeto inicial, isso já é o suficiente para começar de forma correta.

Glossário de termos técnicos

API REST

É uma forma de organizar a comunicação entre cliente e servidor usando HTTP. Em geral, o cliente envia requisições e o servidor devolve respostas em formatos como JSON.

Controller

É o componente que recebe a requisição HTTP, interpreta os dados de entrada, chama a lógica apropriada e devolve a resposta HTTP.

Requisição HTTP

É a mensagem enviada pelo cliente para o servidor. Pode conter método, URL, headers, corpo e outros dados.

Resposta HTTP

É a mensagem devolvida pelo servidor ao cliente. Normalmente contém código de status, headers e corpo da resposta.

Endpoint

É um endereço da API que representa uma funcionalidade específica, como `/login` ou `/logout`.

JSON

É um formato de texto muito usado para troca de dados em APIs. Organiza informações em pares de chave e valor.

Body

É o corpo da requisição. Nele costumam vir os dados principais enviados pelo cliente, como email e senha.

Header

É um conjunto de metadados enviados junto da requisição ou da resposta HTTP. Pode conter, por exemplo, informações de autenticação.

Status HTTP

É o código numérico devolvido pelo servidor para indicar o resultado da operação, como 200, 400, 401 ou 500.

Validação

É a verificação dos dados recebidos para saber se eles estão presentes e em formato aceitável.

Regra de negócio

É uma regra que pertence ao funcionamento do sistema, e não ao protocolo HTTP. Exemplo: decidir se um usuário pode entrar ou não no sistema.

Caso de uso

É a parte da aplicação que executa uma ação com significado de negócio, como autenticar um usuário.

Autenticação

É o processo de verificar a identidade de alguém. Exemplo: confirmar se email e senha correspondem a um usuário válido.

Autorização

É o processo de verificar se um usuário autenticado tem permissão para acessar determinado recurso.

Credenciais

São os dados usados para autenticar um usuário, como login e senha.

Token

É um valor gerado pelo sistema para representar uma sessão autenticada ou autorização de acesso.

Bearer Token

É um tipo de token enviado normalmente no header `Authorization`. Quem possui esse token pode, em muitos casos, acessar recursos protegidos.

Cookie

É um pequeno dado armazenado no navegador e enviado automaticamente em requisições futuras ao servidor. Pode ser usado para sessão ou autenticação.

Sessão

É o estado associado a um usuário autenticado durante seu uso do sistema.

Cache

É um mecanismo de armazenamento temporário para acelerar respostas. Em autenticação, deve ser tratado com cuidado para não armazenar informações sensíveis de forma inadequada.

Log

É o registro de eventos do sistema. Logs ajudam a diagnosticar problemas, mas não devem expor senhas ou dados sensíveis.

Framework

É uma ferramenta que fornece estrutura pronta para desenvolvimento. No caso de uma API em Node.js, Fastify é um exemplo de framework.

Fastify

É um framework para construir APIs HTTP em Node.js, conhecido por simplicidade e desempenho.

DTO

Sigla para Data Transfer Object. É um objeto usado para transportar dados entre partes do sistema.

Camada de interface

É a parte do sistema que lida com entrada e saída, como HTTP, controllers e rotas.

Domínio

É a parte do sistema onde ficam os conceitos e regras principais do problema que está sendo resolvido.

Infraestrutura

É a camada relacionada a banco de dados, framework, serviços externos, filas, arquivos e outros detalhes técnicos.